Robotic Manipulation of Deformable Rope-Like Objects Using Differentiable Compliant Position-Based Dynamics

Fei Liu, *Member, IEEE*, Entong Su, Jingpei Lu, *Graduate Student Member, IEEE*, Mingen Li, and Michael C. Yip, *Senior Member, IEEE*

***Abstract*—Robot manipulation of rope-like objects is an interesting problem with some critical applications, such as autonomous robotic suturing. Solving for and controlling rope is difficult due to the complexity of rope physics and the challenge of building fast and accurate models of deformable materials. While more data-driven approaches have become more popular for finding controllers that learn to do a single task, there is still a strong motivation for a model-based method that could solve many optimization problems. Towards this end, we introduced compliant position-based dynamics (XPBD) to model rope-like objects. Using geometric constraints, the model can represent the coupling of shear/stretch and bend/twist effects. Of crucial importance is that our formulation is differentiable, which can solve parameter estimation problems and improve the matching of rope physics to real-life scenarios (i.e., the real-to-sim problem). For the generality of rope-like objects, two different solvers are proposed to handle the inextensible and extensible effects of varied material stiffness for the rope. We demonstrate our framework's robustness and accuracy on real-to-sim experimental setups using the Baxter robot and the da Vinci research kit (DVRK) (D'Ettorre et al., 2021). Our work leads to a new path for robotic manipulation of the deformable rope-like object taking advantage of the ready-to-use gradients.**

***Index Terms*—Deformable linear objects (DLOs), deformable objects modeling, robotic manipulation, differentiable simulation, sim-to-real transfer, optimizationbased dynamics.**

I. Introduction

Modeling and simulation of deformable objects have recently been of considerable interest in many robotic applications [2], [3], [4]. This includes grasping soft objects, [5] to manipulating soft tissue [6], [7], cloth [8], and even fluids [9], [10]. Among soft structures, deformable linear objects (DLOs), including rope-like objects, strings, cables, flexible beams, etc., are studied (cable routing [11], wire insertion [12], flexible rope [13], and knotting of surgical thread [14]). Recently, techniques involving visual servoing [15], latent dynamics learning [16], and adaptive estimation [17], have been of interest to the community for controlling DLOs.

Classical approaches to manipulation involve model-based controllers, which involve modeling rope as a mechanical structure. Cosserat rod theory [18] is a go-to approach that involves an analytical, partial differential equation (PDE) representing the rope as a continuous 3D-dimensional curve. However, the analytical dynamics are limited by the computational efficiency and stability of solving this PDE with two-point boundary conditions (given by the start and end of the rope).

With regard to simulation, a number of approaches are available for DLOs [19], [20], [21]. Because these simulators generally try to solve the lofty goal of simulating soft bodies for data-driven learning, their focus tends to be on data generation and lacks a method for integration in a model-based control context. In general, many existing robotics simulators only support rigid body simulation and gradient calculations based on finite differences. However, differentiable physics and simulation have become critical features for the robot learning community [22]. Some works [23], [24], [25], [26] have applied differentiable frameworks to rigid articulated body simulation, while others have done the same for deformable objects [27], [28], [29]. These methods provide gradients within the differentiable framework, where optimization tasks for modeling and control are natively supported. Meanwhile, the differentiability can be easily deployed into neural layers for learning-based methods, such as in [30], [31], [32]. Despite these works' attempts to build differentiable models, they were focused on general deformable objects using dynamics such as implicit FEM [28], Material Point Method (MPM) [29], and latent space representations [32]. However, they are limited to modeling and manipulating DLOs due to their specific spatial deformation, i.e., stretching, bending, and twisting.

In the letter, we propose a solution to formulate a differentiable model-based framework for rope-like objects by aiming to fill the gaps mentioned above. The compliant position-based dynamics (XPBD) [33] formulation takes advantage of its focus on computational efficiency, which is of high importance to control and learning applications. We describe the rope with a particle-based object within the XPBD style but in such a manner that automatic differentiation may be leveraged, e.g.,

Manuscript received 10 November 2022; accepted 13 March 2023. Date of publication 5 April 2023; date of current version 22 May 2023. This letter was recommended for publication by Associate Editor P. Di Lillo and Editor J. P. Desai upon evaluation of the reviewers' comments. This work was supported in part by NSF CAREER under Grant #2045803 and in part by the US Army Telemedicine and Advanced Technologies Research Center. *(Fei Liu and Entong Su contributed equally to this work.) (Corresponding author: Fei Liu.)*

The authors are with the Advanced Robotics and Controls Lab, University of California San Diego, La Jolla, CA 92093 USA (e-mail: f4liu@ucsd.edu; ensu@ucsd.edu; jil360@ucsd.edu; mil025@ucsd.edu; yip@ucsd.edu).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2023.3264766, provided by the authors.

Digital Object Identifier 10.1109/LRA.2023.3264766

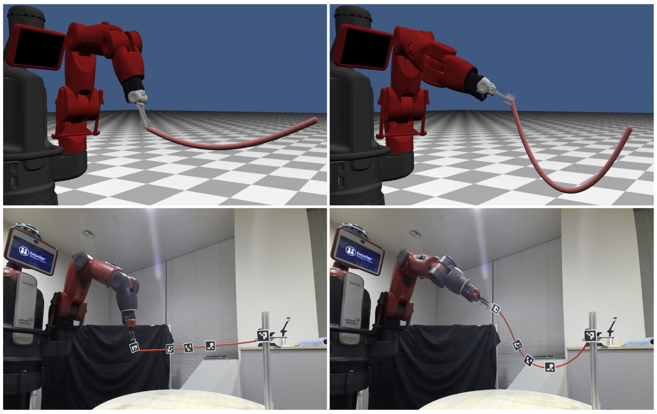

Fig. 1. Simulated and live rope manipulation with matched behavior based on a differentiable compliant position-based dynamics model | The top row shows the simulation result for our shape control task of inextensible rope on the Baxter, rendered in the OpenGL. And the bottom rope is our physical result for this control task in the real scene with learned parameters via differentiable position-based dynamics.

from PyTorch, or adjoint methods. The XPBD modeling for DLOs can guarantee stable forward simulation, while the back-propagation of losses can be applied for parameter identification and control tasks. To the best of our knowledge, we are the first in the literature to benefit from integrating differentiability with analytical models to achieve natural model updates via gradient updates, with a particular application for the robotic manipulation of rope-like objects. The contributions of this letter are as follows:

- We introduced a framework for modeling and simulation for rope-like objects with different stiffness by fitting into a geometrical constraints-based dynamics solver, i.e., XPBD. Because of this, adding other positional constraints makes it simple to seek extensibility.
- In a real-to-sim context, we defined the problem of parameter identification and manipulation of rope-like objects (see Fig. 1). In contrast to data-driven methods [34], [35] that require tuning for the neural parameter, our modeling parameters identified through differential computation are directly linked to the physical properties of the rope objects, i.e., constraint stiffness. As a result, it easily fits into any optimization method.
- We validate our methods in real-world experiments using robot manipulators (Baxter) and surgical robots (dVRK) for different types of ropes. To improve convergency performance and robustness, a combination of direct-linear and iterative algorithms is proposed.

II. Methods

A. XPBD Modeling of Rope-Like Object

The XPBD method will be a foundation for building the differentiable model for DLOs. We firstly discretize the DLO into a sequence of particles (Fig. 2) with Cartesian coordinates $\mathbf{x} \in \mathbb{R}^3$. Meanwhile, quaternions are used to describe orientations *in-between* adjacent particles $\mathbf{q} = [q_w, \mathbf{q}_v] \in \mathbf{SO}(3)$, and will be used to solve the bending and twist deformation of the DLO. Unlike force-based methods such as Euler-Bernoulli beam or Cosserat rod theory, full strain and torsion deformations can be updated with a PBD solver. A detailed description of PBD solvers can be found on [36]. Instead, the position-based dynamics updates position evolution directly using the solution to a quasi-static problem. As a result, position-based approaches omit the velocity layer, which ensures fast, stable, and controllable simulation. Furthermore, it is simple to seek extensibility for task space manipulation of DLOs due to the geometry-based constraints. For brevity, in this letter, we will describe only those aspects of the solver that are important to achieve differentiability for DLOs, i.e., the gradients associated with each definition of a constraint.

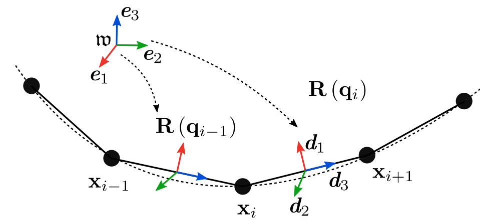

Fig. 2. Proposed geometric model of rope-like objects | The discrete particles include an orientation representation of the rope-like object for modeling with XPBD.

As with all simulations using PBDs, the method starts with a list of constraints, $\mathbf{C}(\mathbf{x} + \Delta\mathbf{x}, \mathbf{q} \oplus \Delta\mathbf{q})$ that describe the dynamics of particles. Solving the constraints involves updating $\Delta\mathbf{x}$ and $\Delta\mathbf{q}$ directly using a non-linear projected Jacobi method used for constrained optimization problems. The method to solve for the gradient updates is derived below.

Constraints can be linearized by Taylor series expansion,

$$\begin{aligned}&\mathbf{C}(\mathbf{x} + \Delta\mathbf{x}, \mathbf{q} \oplus \Delta\mathbf{q})\\ &\approx \mathbf{C}(\mathbf{x}, \mathbf{q}) + \nabla_{\mathbf{x}}\mathbf{C}(\mathbf{x}, \mathbf{q})\Delta\mathbf{x} + \nabla_{\mathbf{q}}\mathbf{C}(\mathbf{x}, \mathbf{q})\Delta\mathbf{q} = 0 \end{aligned} \tag{1}$$

with

$$\begin{aligned}\Delta\mathbf{x} &= \mathbf{W}_{\mathbf{x}}\nabla_{\mathbf{x}}^{\top}\mathbf{C}(\mathbf{x}, \mathbf{q})\Delta\boldsymbol{\Lambda}\\ \Delta\mathbf{q} &= \mathbf{W}_{\mathbf{q}}\nabla_{\mathbf{q}}^{\top}\mathbf{C}(\mathbf{x}, \mathbf{q})\Delta\boldsymbol{\Lambda}\end{aligned} \tag{2}$$

where the Lagrange multiplier change $\Delta\boldsymbol{\Lambda}$ can be found by introducing the compliance parameter $\boldsymbol{\alpha}$ [33],

$$\Delta\boldsymbol{\Lambda} = -\left(\sum_{\pi\in\{\mathbf{x},\mathbf{q}\}} \nabla_{\pi}\mathbf{C}\mathbf{W}_{\pi}\nabla_{\pi}^{T}\mathbf{C} + \boldsymbol{\alpha}\right)^{-1}(\mathbf{C} + \boldsymbol{\alpha}\boldsymbol{\Lambda}) \tag{3}$$

where $\mathbf{W}_{\mathbf{x}}$ and $\mathbf{W}_{\mathbf{q}}$, represented as $\mathbf{W}_{\boldsymbol{\pi}}$ for simplicity, are the weighted terms to ensure linear and angular momentum conservation. Generally, it refers to the mass/inertia matrix as $\mathbf{W}_{\mathbf{x}} = \mathrm{diag}(m_1^{-1} \cdot \mathbf{1}, m_2^{-1} \cdot \mathbf{1}, \ldots, m_N^{-1} \cdot \mathbf{1})$ and $\mathbf{W}_{\mathbf{q}} = \mathrm{diag}(\mathbf{I}_1^{-1}, \mathbf{I}_2^{-1}, \ldots, \mathbf{I}_N^{-1})$. For simplification of DLOs, we use uniformed scalar weights instead of matrices for each dimensionality, which has little impact on simulation and dynamical performance but reduces computational load:

$$\mathbf{W}_{\mathbf{x}} = m_{\mathbf{x}}^{-1} \quad \mathbf{W}_{\mathbf{q}} = \mathbf{I}_{\mathbf{q}}^{-1} \tag{4}$$

Next, we will introduce several geometrical constraints to simulate the behaviors of the DLOs.

1) Shear and Stretch Constraint: According to Cosserat's theory, the shear and stretch measure the deformation regarding the tangent direction of the rope-like object. Therefore, the stretch/compressed length should be constrained relative to its rest pose, which indicates in-extensible elasticity. Simultaneously, the normal direction (i.e., the rotated e_3 from the world frame noted by $\mathfrak{w}$) for each cross-section should be parallel to the tangent direction of the object's centerline, see Fig. 2. It measures the shear strain with respect to non-deformed states. Thus, for each pair of neighboring particles, the shear-stretch deformation can be integrated into a generalized constraint as, $\mathbf{C}^{\mathcal{S}}(\mathbf{x}, \mathbf{q}) = \{\mathbf{c}_i^{\mathcal{S}}(\mathbf{x}_i, \mathbf{x}_{i+1}, \mathbf{q}_i) \mid i \in [1, 2, \ldots, N-1]\}$, which is,

$$\mathbf{c}_i^{\mathcal{S}}(\mathbf{x}_i, \mathbf{x}_{i+1}, \mathbf{q}_i) = \frac{\mathbf{x}_{i+1} - \mathbf{x}_i}{\|\bar{\mathbf{x}}_{i+1} - \bar{\mathbf{x}}_i\|} - \mathbf{R}(\mathbf{q}_i)\, \boldsymbol{e}_3 \tag{5}$$

where $\mathbf{R}(\mathbf{q}_i)$ is the rotation matrix from the local frame of i^{th} line segment to world frame $\mathfrak{w}$ represented using quaternion, $\bar{\cdot}$ represent the states in rest pose. According to the gradient calculation in [37], we can obtain,

$$\begin{aligned}
\nabla_{\mathbf{x}_i}^{\top} \mathbf{c}_i^{\mathcal{M}} &= -\|\bar{\mathbf{x}}_{i+1} - \bar{\mathbf{x}}_i\|^{-1}\, \mathbf{1}_{3\times 3} \\
\nabla_{\mathbf{x}_{i+1}}^{\top} \mathbf{c}_i^{\mathcal{S}} &= \|\bar{\mathbf{x}}_{i+1} - \bar{\mathbf{x}}_i\|^{-1}\, \mathbf{1}_{3\times 3} \\
\nabla_{\mathbf{q}_i}^{\top} \mathbf{c}_i^{\mathcal{S}} &= \nabla_{\mathbf{q}_i}^{\top} [\mathbf{R}(\mathbf{q}_i)\, e_3] = 2(q_{i,w} \boldsymbol{e}_3 - \boldsymbol{e}_3 \times \mathbf{q}_{i,v} | \mathbf{q}_{i,v}^{\top} \boldsymbol{e}_3 \\
&\quad + \mathbf{q}_{i,v} \boldsymbol{e}_3^{\top} - \boldsymbol{e}_3 \mathbf{q}_{i,v}^{\top} - \mathbf{q}_{i,w}^{\top} [e_3]^{\times})
\end{aligned} \tag{6}$$

where $[\cdot]^{\times}$ is the skew-symmetrical matrix of a vector, and $|$ is the divider for a $(3 \times 1 | 3 \times 3)$ matrix.

2) Bend and Twist Constraint: In differential geometry, the Darboux vector $\boldsymbol{\Omega}$ is used to parameterize strain deformation with respect to frame rotation. According to Cosserat theory, the Darboux vector can be expressed as a quaternion by measuring the rod's twist in the tangent direction. Thus, the difference between the current and resting configuration should be evaluated, i.e. $\boldsymbol{\Omega} - \bar{\boldsymbol{\Omega}}$. According to this [37], the bend and twist constraint can be computed for each pair of two adjacent quaternions (shown in Fig. 2) by, $\mathbf{C}^{\mathcal{B}}(\mathbf{q}) = \{\mathbf{c}_i^{\mathcal{B}}(\mathbf{q}_i, \mathbf{q}_{i+1}) \mid i \in [1, 2, \ldots, N-1]\}$, which is,

$$\begin{aligned}
\mathbf{c}_i^{\mathcal{B}}(\mathbf{q}_i, \mathbf{q}_{i+1}) &= \boldsymbol{\Omega} - \xi \cdot \bar{\boldsymbol{\Omega}} = \mathtt{Im}(\mathbf{q}_i^* \cdot \mathbf{q}_{i+1} - \bar{\mathbf{q}}_i^* \cdot \bar{\mathbf{q}}_{i+1}) \\
\xi &= \mathtt{sign}(\boldsymbol{\Omega} + \bar{\boldsymbol{\Omega}})
\end{aligned} \tag{7}$$

where $\mathtt{Im}(\cdot)$ is the imaginary part of the quaternion and $(\cdot)^*$ is the conjugate quaternion. The constraint gradients are then

$$\begin{aligned}
\nabla_{\mathbf{q}_i}^{\top} \mathbf{c}_i^{\mathcal{B}} &= -\left(-\mathbf{q}_{i+1,v} \mid q_{i+1,w} \mathbf{1}_{3\times 3} - [\mathbf{q}_{i+1,v}]^{\times}\right) \\
\nabla_{\mathbf{q}_{i+1}}^{\top} \mathbf{c}_i^{\mathcal{B}} &= +\left(-\mathbf{q}_{i,v} \mid q_{i,w} \mathbf{1}_{3\times 3} - [\mathbf{q}_{i,v}]^{\times}\right)
\end{aligned} \tag{8}$$

3) Distance Constraint: One property of modeling ropes is that they can be considered either extensible or inextensible based on how stiff they are, and ultimately this can be modeled as a constraint. Although the above strain deformation has considered the inextensible property implicitly, the iterative solver in (2) cannot guarantee that all the constraints will be satisfied. Therefore, we can explicitly consider the distance constraint for an enhanced inextensible chain structure using only discretized particle positions. We define $\mathbf{C}^{\mathcal{D}}(\mathbf{x}) = \{\mathbf{c}_i^{\mathcal{S}}(\mathbf{x}_i, \mathbf{x}_{i+1}) \mid i \in [1, 2, \ldots, N-1]\}$ by,

$$\mathbf{c}_i^{\mathcal{D}}(\mathbf{x}_i, \mathbf{x}_{i+1}) = \|\mathbf{x}_{i+1} - \mathbf{x}_i\| - \|\bar{\mathbf{x}}_{i+1} - \bar{\mathbf{x}}_i\| \tag{9}$$

The gradients can be easily obtained by,

$$-\nabla_{\mathbf{x}_i}^{\top} \mathbf{c}_i^{\mathcal{D}} = \nabla_{\mathbf{x}_{i+1}}^{\top} \mathbf{c}_i^{\mathcal{D}} = \frac{\mathbf{x}_{i+1} - \mathbf{x}_i}{\|\bar{\mathbf{x}}_{i+1} - \bar{\mathbf{x}}_i\|} \tag{10}$$

B. Real-to-Sim Parameter Identification

Given the above constraints, the dynamics of a rope-like object can be represented by a set of discretized particles (shown in Fig. 2) with position and orientation evolution. However, it is just an approximation of the real model for the rope objects. For the above shear/shear, bend/twist and distance constraints, we can introduce additional stiffness parameters to weight the updates during iteration steps, i.e.,

$$\boldsymbol{\eta}^* = \{\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{S}}, \boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{S}}, \boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{B}}, \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{D}}, \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}}, \boldsymbol{\eta}^{\mathcal{SOR}}\} \tag{11}$$

which stands for shear/stretch, bend/twist, and distance constraints regarding position or orientation. Moreover, $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}}$ stands for position changes due to external gravity, and $\boldsymbol{\eta}^{\mathcal{SOR}}$ stands for the successive over-relaxation parameter and is applied to accelerate the convergence speed further.

C. Differential Framework

To perform an optimal control task, the ready-to-use gradients will be needed to compute. We introduce the XPBD inside a differentiable framework, as shown in Algorithm 1 .[1] We rely on the automatic differentiation function provided by PyTorch to obtain the gradients. It can be natively integrated into optimization or learning methods since the coding framework can be viewed as a differentiable layer supporting forward and backpropagation operations. The computing memory might be limited by the number of iterations for substep simulation. Denoting the gradient variable at time t by $\boldsymbol{\lambda}^t$, we can formulate the following optimization problem,

$$\begin{aligned}
\boldsymbol{\lambda}^t &= \operatorname{argmin} \mathcal{L}(\mathbf{x}, \mathbf{q}) \\
\text{s.t.}\ \ &\mathbf{C}^{\mathcal{S}}(\mathbf{x}, \mathbf{q}) = 0 \\
&\mathbf{C}^{\mathcal{B}}(\mathbf{q}) = 0 \\
&\mathbf{C}^{\mathcal{D}}(\mathbf{x}) = 0
\end{aligned} \tag{12}$$

where $\mathbf{C}^{\mathcal{S}}$, $\mathbf{C}^{\mathcal{A}}$ and $\mathbf{C}^{\mathcal{D}}$ are the position-based constraints, and $\mathcal{L}$ is the loss function derived from the system states. $\boldsymbol{\lambda}^t$ represents the selected gradient variable, which can be control states, system parameters, or system states.

III. Real-to-Sim and the Shape Control Problem

We used the proposed differentiable framework to conduct three experiments on Rethink Baxter and the da Vinci Research Kit (dVRK) robotic platforms in this part. The Baxter represents

[1]In this letter, we only consider the quasi-static dynamical states. Only gravity will be regarded as without any external torques. Thus, the Euler prediction and integration of velocity and angular speed will not be involved.

Algorithm 1: Differentiable Framework for Rope-like Objects.

// Initialize the gradients variable
$\boldsymbol{\lambda}^t \leftarrow \{\mathbf{x}^t, \mathbf{q}^t, \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{S}}, \boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{S}}, \boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{B}}, \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{D}}, \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}}, \boldsymbol{\eta}^{\mathcal{SOR}}, \cdots\cdots\}$
// Position states Euler prediction
$\mathbf{x}^{t+1} \leftarrow \mathbf{x}^t + \frac{1}{2}\mathbf{g}\Delta t^2 \cdot \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}}$
// Constraints solving loop
while *iter* $<$ *iterations* **do**
  // Apply shear/stretch constraints (Eq. 5)
  $\Delta\mathbf{x}^{\mathcal{S}}, \Delta\mathbf{q}^{\mathcal{S}} \leftarrow \texttt{solveShearStretch}\left(\mathbf{C}^{\mathcal{S}}\left(\mathbf{x}^{t+1}, \mathbf{q}^{t+1}\right) = 0\right)$
  // Apply bend/twist constraints (Eq. 7)
  $\Delta\mathbf{q}^{\mathcal{B}} \leftarrow \texttt{solveBendTwist}\left(\mathbf{C}^{\mathcal{B}}\left(\mathbf{q}^{t+1}\right) = 0\right)$
  // Apply distance constraints (Eq. 9)
  $\Delta\mathbf{x}^{\mathcal{D}} \leftarrow \texttt{solveDistance}\left(\mathbf{C}^{\mathcal{D}}\left(\mathbf{x}^{t+1}\right) = 0\right)$
  // Update constraints changes
  $\mathbf{x}^{t+1} \leftarrow \mathbf{x}^{t+1} + \left(\Delta\mathbf{x}^{\mathcal{S}} \cdot \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{S}} + \Delta\mathbf{x}^{\mathcal{D}} \cdot \boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{D}}\right)/2 * \boldsymbol{\eta}^{\mathcal{SOR}}$
  $\mathbf{q}^{t+1} \leftarrow \mathbf{q}^{t+1} + \left(\Delta\mathbf{q}^{\mathcal{S}} \cdot \boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{S}} + \Delta\mathbf{q}^{\mathcal{B}} \cdot \boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{B}}\right)/2 * \boldsymbol{\eta}^{\mathcal{SOR}}$
// Obtain the loss function
$\mathcal{L} \leftarrow \mathcal{L}\left(\mathbf{x}^{t+1}, \mathbf{q}^{t+1}\right)$
// Calculate the gradients
$\partial\mathcal{L}/\partial\boldsymbol{\lambda}^t = \texttt{autodiff}(\mathcal{L})$
return $\partial\mathcal{L}/\partial\boldsymbol{\lambda}^t$

a situation of more significant rope manipulation, whereas the dVRK represents a situation of surgical automation involving blood vessel manipulation. We considered the Baxter-rope experiment to involve an inextensible DLO, while the dVRK blood-vessel experiment to be an extensible DLO. For both setups, we looked at both a real-to-sim problem (i.e., parameter estimation based on observations from the real world) and a control problem (i.e., using the XPBD model we developed to have rope configuration reach a target configuration iteratively). The implementation of each experiment will be provided later.

A. Solver Setups

In a classical PBD solver [36], the Jacobi approach averages each constraint step changes and keeps updating iteratively. The convergence compromises the number of iterations and the satisfaction of other constraints. However, it was unsuitable for simulating the inextensible effects of ropes with significant axial stiffness.

Precisely, we needed to guarantee the fulfillment of the distance constraint in (9) completely. A direct-linear solver was proposed [38] based on the tridiagonal matrix algorithm, i.e., the Thomas algorithm, to preserve the inextensible characteristics of rope-like objects. We will refer to methods individually as the Jacobi XPBD and Thomas XPBD, respectively. All other constraints, such as shear/stretch and bend/twist constraints, used the Jacobi method.[2]

B. Loss Functions

The rope was simulated with the particle-based method and represented by connected lines. We defined Point-to-Line loss as the primary loss to investigate for shape matching. We intended to find the minimum distance from each preprocessed real data point $\mathbf{p}_j$ from a point cloud or 2D centerline, to the line segments L_i between each pair of adjacent simulated particle $(\mathbf{x}_i, \mathbf{x}_{i+1})$. Thus, the primary loss regarding point-to-line loss was

$$\mathcal{L}_{PL} = \sum_{j=1..N} \min_{\tilde{\mathbf{p}} \in \bigcup_{k=1}^{K-1} L_k} \left\|\mathbf{p}^j - \tilde{\mathbf{p}}\right\| \tag{13}$$

where we discretized the rope to K particle nodes, along N preprocessed real data points, and $\tilde{\mathbf{p}}$ was the projected locations on each line segment with minimum distance. In our experiments, the point-to-line loss could be applied to either the 3D point cloud (OBJ1) or 2D centerline (OBJ2).

[2]Thus, in this letter, when Jacobi XPBD and Thomas XPBD are indicated, it is distinguishing how the distance constraint is solved.

TABLE I
PARAMETER ESTIMATION RESULT FOR INEXTENSIBLE ROPE WITH THE DIFFERENTIABLE THOMAS XPBD SOLVER

Parameter	Initial	Optimized	Meaning
$\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}}$	0.04	0.024	position stiffness (gravity)
$\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{D}}$	1.0	0.48	position stiffness (distance)
$\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{S}}$	1.0	1.16	position stiffness (shear/stretch)
$\boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{S}}$	1.0	0.52	quaternion stiffness (shear/stretch)
$\boldsymbol{\eta}_{\mathbf{q}}^{\mathcal{B}}$	1.0	1.49	quaternion stiffness (bending/twist)
$\boldsymbol{\eta}^{\mathcal{SOR}}$	1.0	0.61	successive over-relaxation weight

IV. Experiments and Result Analysis

A. Inextensible Rope Parameter Identification and Optimal Particle Numbers

Parameter identification and position estimation of the control point were carried out on an inextensible rope manipulated by the Baxter robot. We fixed one endpoint of the rope to the environment and the other endpoint of the rope to the manipulator end-effector (referred to as the control point). We continuously moved the control point and collected 36 frames of point cloud for the rope deformation. The Microsoft Azure Kinect was used to obtain the 2D RGB image frames for centerline extraction at the same time. Since only translational movement (fixed rotation angle) of the end-effector is controlled, the point cloud had occlusions and was not fully visible at the local region near the control point. We applied the keypoint-based kinematics reconstruction [39] to identify the 3D position of the control point. Therefore, the ground truth data for the 3D point cloud and 2D centerline, as well as the control point, we're ready for real-to-sim transfer.

In XPBD simulation, considering the computational cost and stiffness of the rope, the number of particle were set by $K = 20$. The particle $\mathbf{x}_0$ was the control point and $\mathbf{x}_{19}$ corresponding to the endpoint was fixed to the environment. According to Section II-B, six constraint stiffness parameters were needed to estimate, as shown in Table I. Because 3D point cloud could be more informative than 2D centerline which lacked depth data, the parameters were optimized with `OBJ1` using the 3D point-to-line loss. For parameter identification, there are six parameters needed to be estimated for the solver, as shown in

TABLE III
PARAMETER ESTIMATION RESULT OF DIFFERENTIABLE JACOBI SOLVER FOR INEXTENSIBLE AND EXTENSIBLE ROPE

Parameter	Inextensible rope	extensible rope
$\eta_{\mathbf{x}}^{\mathcal{G}}$	0.26(bending) / 0.001(tensioned)	25(relax) / 3(tensioned)
$\eta_{\mathbf{x}}^{\mathcal{D}}$	1.19	0.87
$\eta_{\mathbf{x}}^{\mathcal{S}}$	0.80	0.873
$\eta_{\mathbf{q}}^{\mathcal{S}}$	1.19	1.0
$\eta_{\mathbf{q}}^{\mathcal{B}}$	0.80	1.30
$\eta^{\mathcal{SOR}}$	0.79	0.61

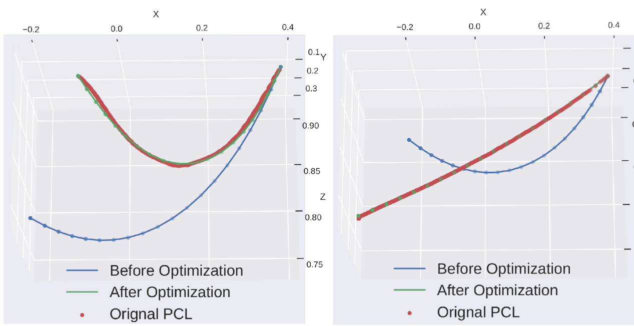

Fig. 3. Parameter optimization results for inextensible rope (i.e., real-to-sim result) | The bending (left) and the tensioned (right) status of the simulated rope before and after parameter optimization. PCL, in red, is the real point cloud to be matched.

TABLE II
THE CONVERGENCE TIME, ITERATION AND LOSS OF AN INEXTENSIBLE ROPE BY USING THE DIFFERENT NUMBER OF PARTICLES TO SIMULATE BASED ON THE JACOBI SOLVER IN SIMULATION

Number of Particles	30	20	10
Converg. Time(s)	74.68	8.43	6.71
Converg. Iterations	≈350	≈50	≈50
Converg. Loss(mean/std)	8.16/3.46	7.24/3.46	10.92/2.94

(11). After parameter optimization, the simulation states were approximately approaching the ground truth point cloud. The differentiable Jacobi solver needs two sets of gravity stiffness to deal with different distance constraints for varied rope states (bending and tensioned), as shown in Table III. They can be determined by the user-specified control frames. The comparison of simulated results before and after parameter optimization is shown in Fig. 3. As [36] indicated that the parameters in Table I are related to the simulation step and the rope's elasticity. Thus, by optimizing these parameters in Tables I and III, both the Jacobi solver and Thomas XPBD solver can simulate different rope states.

We conducted an experiment to determine the optimal number of particles in the simulation, which is another critical parameter. Table II shows the convergence of using different numbers of particles based on Jacobi XPBD. The computation overhead increases significantly as the number of particles increases. We found that discretizing the rope into 10 or 20 particles improves convergence performance while using 30 particles requires 350 iterations and still fails to achieve the performance compared to using 20 particles. Even though using 10 particles is time and computation efficient, such a model is not flexible enough to simulate the geometry of the actual rope and results in a larger loss. Thus, we decide to use 20 particles for the experiments.

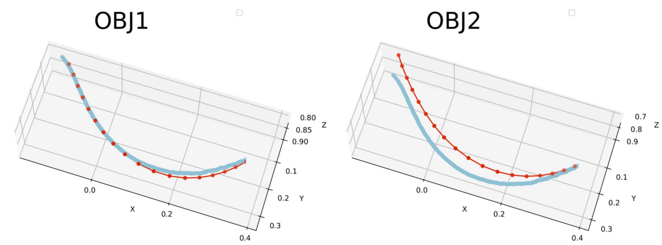

Fig. 4. Deformation estimation for inextensible rope using different loss functions | The simulation result for 2 different loss functions (OBJ1 and OBJ2) based on the Thomas XPBD solver. Blue represents ground truth provided by the point clouds. Red represents the optimized result regarding different loss functions.

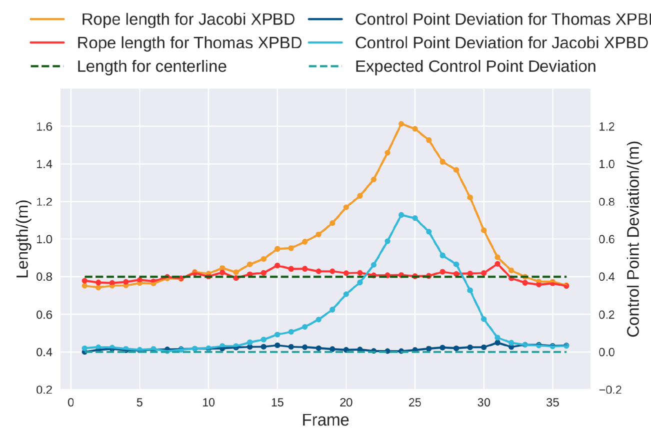

Fig. 5. Comparison of Thomas and Jacobi XPBD solver on inextensible rope | Control point deviation and rope length regarding different frames of simulation using both solvers.

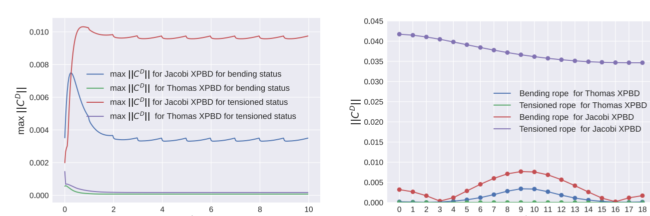

Fig. 6. Simulation result analysis for distance constraint of inextensible rope on Baxter | (1) The evolution of maximal distance constraint value of all nodes (i.e., max$\|\mathbf{C}^{\mathcal{D}}\|$) regarding iterations. (2) The $\|\mathbf{C}^{\mathcal{D}}\|$ value of each pair of neighboring nodes of the last iteration. Both the tensioned and bending status are shown based on the differentiable Thomas solver and Jacobi solver of XPBD simulation ($\eta_{\mathbf{x}}^{\mathcal{G}} = 0.26$ for Jacobi XPBD solver.)

The 3D point cloud included the rope information in 3D space, while the 2D centerline did not contain depth information. As a result, the simulation result containing the 3D information showed the best performance (Fig. 4).

We also compared the performance of Thomas XPBD and Jacobi XPBD when estimating the position of the control point. The average length deviation over the ground truth length for the Thomas XPBD solver was 2% for all structures, but for the

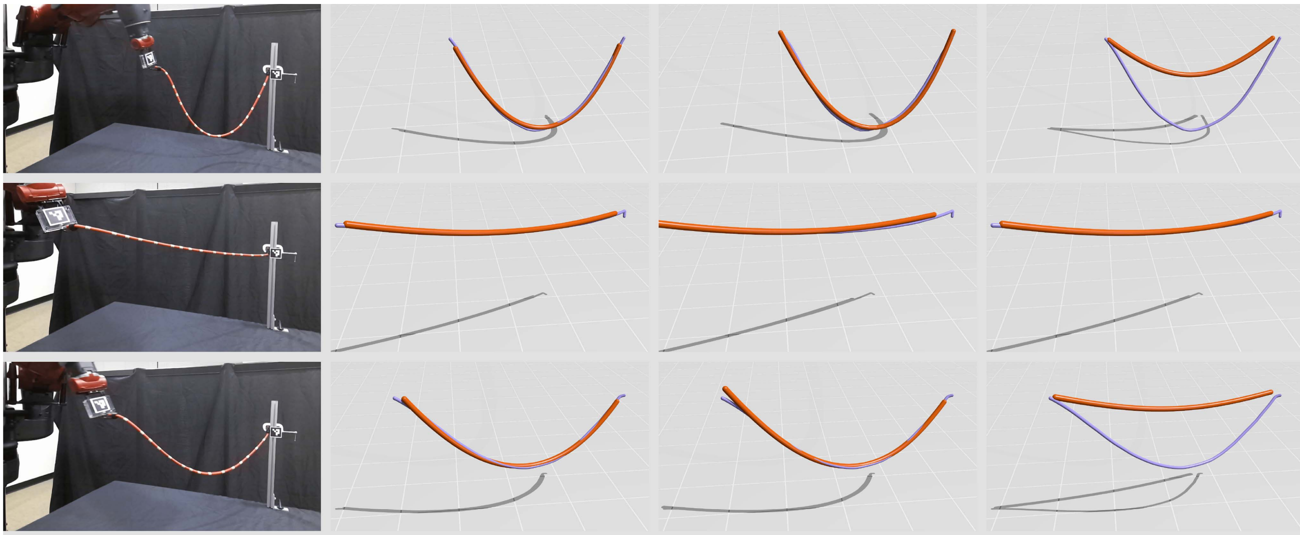

Fig. 7. Deformation estimation for inextensible rope on Baxter |Each Column: (1) Raw RGB image. (2) The simulation result is based on Thomas XPBD, with good convergency for both bending and tensioned status. The simulation result with gravity stiffness of (3) $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}} = 0.26$ (better convergency for bending status and the length of tensioned rope in the middle exceeds the normal length) and (4) $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}} = 0.001$ (better convergency for tensioned status and the elasticity of the bending ropes in the top and bottom fail to reach the expected.) from Jacobi XPBD solver. Blue ropes represented ground truth provided by the point clouds. Orange ropes represented the optimized deformation of the rope. For the light occlusion, we used keypoint detection [39] and point cloud instead of Aruco Marker for the location identification of the end-effector and fixed point, respectively.

Jacobi solver, it was 26.6%. Especially when the rope became tense, the error for Jacobi solver became more significant. The same phenomenon occurred on the error of the control point's position in Fig. 5. The left and right plots in Fig. 6 indicated that the distance constraint in the Jacobi XPBD was hard to be satisfied when the rope was approaching the tensioned status. The Jacobi XPBD intuitively tried to comprise the satisfaction of each constraint solving. It iteratively struggled to maintain the stretch, twist, and distance against gravity. Thus, it resulted in a no-complete convergency of distance constraint $\mathbf{C}^{\mathcal{D}}$.

For Thomas XPBD, the distance constraint would be satisfied between each neighboring node simultaneously within one step solving, which was similar as the position update mechanism of the gravity. In this case, the Thomas XPBD solver ensured that the length of the rope was unchanged, and it was more suitable for the simulation of an inextensible rope. The simulation result shown in Fig. 7 proved our conclusion.

B. extensible Rope Parameter Identification and Key Points Estimation

Parameter identification and position estimation of control point were carried out on an extensible, flexible silicone tube using the dVRK surgical robot. The silicone tube was chosen to resemble a compliant and dissected vessel, a technique often used in surgery to avoid damaging the vessel. The two endpoints of the rope were fixed, and the control point was in the middle of the rope. We moved the control point to collect 30 frames of point cloud data. In simulation, the number of particle nodes was $K = 40$. It was defined by the grid search following the same criteria as in Table II. $\mathbf{x}_0$ and $\mathbf{x}_{39}$ corresponding to both the endpoints were fixed and $\mathbf{x}_{20}$ was the control point. Since the silicon rope had a lower axial stiffness, we did not need to preserve the inextensible behavior completely. Thus, we only implemented the Jacobi XPBD solver for the distance constraint in this experiment. The loss function used was `OBJ1`, and we estimated the same parameters as the ones indicated in Section II-B. After parameter estimation, we estimated the position of the control point (i.e. $\boldsymbol{\lambda} = \mathbf{x}_{19}$) using `OBJ1` and `OBJ2`, i.e., point-to-line for both 3D and 2D cases.

For parameter identification, we used grid-search to identify the optimized six parameters in Table III for extensible rope. Since the control point was set at the middle of the rope, manipulation could result in one side being tight and the other being loose. Thus, the effect of the gravity over two sides was different, and we needed to define different gravity weights $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}}$ for the two sides, as shown in Fig. 8.

With the optimized parameters, we estimated the position of the control point and a reference point, namely by key points. Apart from the control point, there was a reference point with a red marker, shown in Fig. 8. The marker was used only to compare ground truth trajectory and simulation results. The average error of the location of reference and control point is 8.869 pixels and 10.74 pixels, respectively, for pixel deviation in the 2D image frame. In Fig. 9, we showed the original image and simulation result for different frames; no matter how the rope was deformed, our solver could get a desirable outcome. Even though the Thomas solver outperformed the Jacobi solver in the previous experiment, Thomas XPBD was too tough to simulate the silicon rope. Thomas XPBD required the length of the rope to remain unchanged, which contradicted the silicon rope's extensible property. Thus, the Thomas solver diverged for the simulation, and the Jacobi solver was more suitable.

Fig. 8. Parameter identification for extensible rope on dVRK | This experiment shows the simulation result with different $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}}$. From left to right: (1) Original RGB image from Azure Kinect. (2) Both sides were $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}} = 3$. (3) Both sides were $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}} = 25$. (4) The relax side was $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}} = 25$ and the extensible (tensioned) side was $\boldsymbol{\eta}_{\mathbf{x}}^{\mathcal{G}} = 3$. The simulation result from (4) was the closest to the ground truth.

Fig. 9. Deformation identification and key points estimation for extensible rope on dVRK | Top: Original RGB image from Azure Kinect. Bottom: Simulation result for the extensible rope using Jacobi XPBD solver. Orange ropes represented the optimized deformation of the rope. The manipulation policy from the differential framework made the simulated result approaches the ground truth. The right red marker is the control point, and the left one is the reference marker.

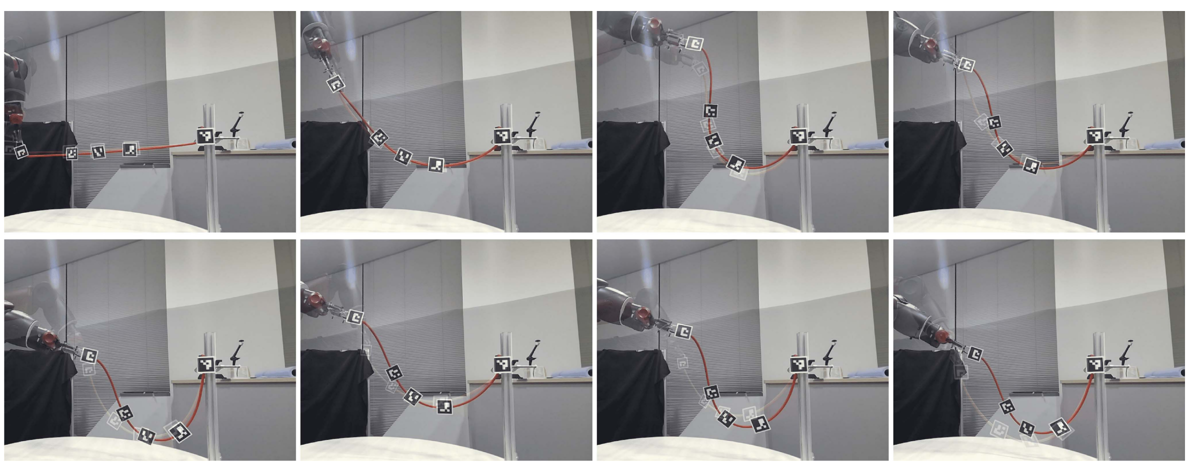

Fig. 10. Shape control for inextensible rope on Baxter | Top Row: The ideal shape control task by identified parameters. Bottom Row: The not-ideal cases. The target control shape for ropes is in low opacity, while our control results are in solid opacity.

C. inextensible Rope Shape Control

Based on the Thomas XPBD solver, the shape control task was carried out on the same Baxter setup as the Section IV-A. The ground truth shape of the rope was obtained by locating the left control point (end-effector), right fixed endpoint, and the middle three Aruco Markers shown [40] in the Figs. 1 and 10. We fixed the simulated rope to the same right endpoint. By setting the middle three points as the shape target, we optimized the position of the control point from the differentiable Thomas XPBD simulation.

As shown in Fig. 10, the transparent rope with low opacity was our shape control target status, and the solid ones with high opacity were the result controlled by the identified parameters from Section IV-A. Our controlled shape could almost overlap with the target rope, as shown in the Top Row of Fig. 10. However, some had deviations as shown in the Bottom Row of Fig. 10. One reason is that we only considered three shape segment points as the target for loss computation. More discretized segments of the ground truth data involved in loss will improve the accuracy. However, if we place more segments that require smaller marker sizes, the perception will be less than ideal due to limited camera resolution. Another reason was because we did not consider the rotational control of the end-effector when solving the inverse kinematics of the Baxter, which caused the rope not to be able to move to the desired position perfectly.

V. Conclusion and Future Works

This letter used a compliant position-based framework to conduct the differential real-to-sim tasks for parameter identification and shape control tasks. Several geometrical constraints were introduced to model the rope-like objects' coupling stretch/shear and bending/twisting effects. To inspect the inextensible and extensible impact, the Thomas solver and the Jacobi solver are proposed for the distance constraint. The experiment results on the Baxter robot and DVRK platform proved the validity and robustness of our solvers. The shape control tasks showed a novel path for real-to-sim robotic manipulation operations.

Future works will consider differentiable control with collision handling and rigid-deformable coupling. Meanwhile, more advanced tasks, such as surgical thread manipulation in field environments using the proposed inextensible solver, will be considered.

Acknowledgment

The authors would like to thank Yutong Zhang for the rendering of the cover photo.

References

[1] C. D'Ettorre et al., "Accelerating surgical robotics research: A review of 10 years with the da Vinci research kit," *IEEE Robot. Automat. Mag.*, vol. 28, no. 4, pp. 56–78, Dec. 2021.

[2] H. Yin, A. Varava, and D. Kragic, "Modeling, learning, perception, and control methods for deformable object manipulation," *Sci. Robot.*, vol. 6, no. 54, 2021, Art. no. eabd8803.

[3] J. Sanchez, J.-A. Corrales, B.-C. Bouzgarrou, and Y. Mezouar, "Robotic manipulation and sensing of deformable objects in domestic and industrial applications: A survey," *Int. J. Robot. Res.*, vol. 37, no. 7, pp. 688–716, 2018.

[4] J. Zhu et al., "Challenges and outlook in robotic manipulation of deformable objects," *IEEE Robot. Automat. Mag.*, vol. 29, no. 3, pp. 67–77, Sep. 2021.

[5] I. Huang et al., "DefGraspSim: Physics-based simulation of grasp outcomes for 3D deformable objects," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 6274–6281, Jul. 2022.

[6] F. Liu, Z. Li, Y. Han, J. Lu, F. Richter, and M. C. Yip, "Real-to-sim registration of deformable soft tissue with position-based dynamics for surgical robot autonomy," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 12328–12334.

[7] Y. Han, F. Liu, and M. C. Yip, "A 2D surgical simulation framework for tool-tissue interaction," in *Proc. IROS Workshop*, 2021.

[8] Y. Wu, W. Yan, T. Kurutach, L. Pinto, and P. Abbeel, "Learning to manipulate deformable objects without demonstrations," in *Proc. Robot.: Sci. Syst.*, 2020.

[9] J. Huang, F. Liu, F. Richter, and M. C. Yip, "Model-predictive control of blood suction for surgical hemostasis using differentiable fluid simulations," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 12380–12386.

[10] C. Schenck and D. Fox, "SPNets: Differentiable fluid dynamics for deep neural networks," in *Proc. 2nd Conf. Robot Learn.*, 2018, pp. 317–335.

[11] D. McConachie, T. Power, P. Mitrano, and D. Berenson, "Learning when to trust a dynamics model for planning in reduced state spaces," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 3540–3547, Apr. 2020.

[12] R. Lagneau, A. Krupa, and M. Marchal, "Automatic shape control of deformable wires based on model-free visual servoing," *IEEE Robot. Automat. Lett.*, vol. 5, no. 4, pp. 5252–5259, Oct. 2020.

[13] F. Zhong, Y. Wang, Z. Wang, and Y.-H. Liu, "Dual-arm robotic needle insertion with active tissue deformation for autonomous suturing," *IEEE Robot. Automat. Lett.*, vol. 4, no. 3, pp. 2669–2676, Jul. 2019.

[14] B. Lu et al., "A learning-driven framework with spatial optimization for surgical suture thread reconstruction and autonomous grasping under multiple topologies and environmental noises," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 3075–3082.

[15] J. Zhu, D. Navarro-Alarcon, R. Passama, and A. Cherubini, "Vision-based manipulation of deformable and rigid objects using subspace projections of 2D contours," *Robot. Auton. Syst.*, vol. 142, May 2021, Art. no. 103798.

[16] W. Zhang, K. Schmeckpeper, P. Chaudhari, and K. Daniilidis, "Deformable linear object prediction using locally linear latent dynamics," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 13503–13509.

[17] M. Yu, H. Zhong, and X. Li, "Shape control of deformable linear objects with offline and online learning of local linear deformation models," in *Proc. Int. Conf. Robot. Automat.*, 2022, pp. 1337–1343.

[18] M. Gazzola, L. Dudte, A. McCormick, and L. Mahadevan, "Forward and inverse problems in the mechanics of soft filaments," *Roy. Soc. Open Sci.*, vol. 5, no. 6, 2018, Art. no. 171628.

[19] R. Laezza, R. Gieselmann, F. T. Pokorny, and Y. Karayiannidis, "ReForm: A robot learning sandbox for deformable linear object manipulation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2021, pp. 4717–4723.

[20] X. Lin, Y. Wang, J. Olkin, and D. Held, "SoftGym: Benchmarking deep reinforcement learning for deformable object manipulation," in *Proc. Conf. Robot Learn.*, 2020, pp. 432–448.

[21] M. Yu, H. Zhong, and X. Li, "Shape control of deformable linear objects with offline and online learning of local linear deformation models," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2022, pp. 1337–1343.

[22] Y. Hu et al., "DiffTaichi: Differentiable programming for physical simulation," in *Proc. Int. Conf. Learn. Representations*, 2020.

[23] F. de Avila Belbute-Peres, K. Smith, K. Allen, J. Tenenbaum, and J. Z. Kolter, "End-to-end differentiable physics for learning and control," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018.

[24] Y.-L. Qiao, J. Liang, V. Koltun, and M. C. Lin, "Efficient differentiable simulation of articulated bodies," in *Proc. 38th Int. Conf. Mach. Learn.*, 2021, vol. 139, pp. 8661–8671.

[25] K. Werling, D. Omens, J. Lee, I. Exarchos, and C. K. Liu, "Fast and feature-complete differentiable physics engine for articulated rigid bodies with contact constraints," in *Proc. Robot., Sci. Syst.*, 2021.

[26] F. Liu, M. Li, J. Lu, E. Su, and M. C. Yip, "Parameter identification and motion control for articulated rigid body robots using differentiable position-based dynamics," 2022, *arXiv:2201.05753*.

[27] J. K. Murthy et al., "gradSim: Differentiable simulation for system identification and visuomotor control," in *Proc. Int. Conf. Learn. Representations*, 2021.

[28] T. Du et al., "DiffPD: Differentiable projective dynamics," *ACM Trans. Graph.*, vol. 41, no. 2, Nov. 2021, Art. no. 13.

[29] Z. Huang et al., "Plasticinelab: A soft-body manipulation benchmark with differentiable physics," in *Proc. Int. Conf. Learn. Representations*, 2021.

[30] Y.-L. Qiao, J. Liang, V. Koltun, and M. Lin, "Differentiable simulation of soft multi-body systems," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 17123–17135.

[31] P. Ma et al., "Diffaqua: A differentiable computational design pipeline for soft underwater swimmers with shape interpolation," *ACM Trans. Graph.*, vol. 40, no. 4, 2021, Art. no. 132.

[32] S. Chen, Y. Liu, S. W. Yao, J. Li, T. Fan, and J. Pan, "DiffSRL: Learning dynamical state representation for deformable object manipulation with differentiable simulation," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 9533–9540, Oct. 2022.

[33] M. Macklin, M. Müller, and N. Chentanez, "XPBD: Position-based simulation of compliant constrained dynamics," in *Proc. 9th Int. Conf. Motion Games*, 2016, pp. 49–54.

[34] P. Sundaresan et al., "Untangling dense non-planar knots by learning manipulation features and recovery policies," in *Proc. Robot.: Sci. Syst.*, 2021.

[35] S. Huo et al., "Keypoint-based planar bimanual shaping of deformable linear objects under environmental constraints with hierarchical action framework," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 5222–5229, Apr. 2022.

[36] M. Macklin, M. Müller, and J. Bender, "Position-based simulation methods in computer graphics," *Eurograph. Tutorial*, 2017.

[37] T. Kugelstadt and E. Schömer, "Position and orientation based cosserat rods," in *Proc. ACM SIGGRAPH/Eurograph. Symp. Comput. Animation*, 2016, pp. 169–178.

[38] L. Xu and Q. Liu, "Real-time inextensible surgical thread simulation," *Int. J. Comput. Assist. Radiol. Surg.*, vol. 13, no. 7, pp. 1019–1035, Jul. 2018.

[39] J. Lu, F. Richter, and M. C. Yip, "Pose estimation for robot manipulators via keypoint optimization and sim-to-real transfer," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 4622–4629, Apr. 2022.

[40] F. J. Romero-Ramirez, R. Muñoz-Salinas, and R. Medina-Carnicer, "Speeded up detection of squared fiducial markers," *Image Vis. Comput.*, vol. 76, pp. 38–47, 2018.